Smart-Chatbot

Outline

1. Web Crawling and Data Scraping

Scrapy Framework

We've implemented a web crawler with the **Scrapy framework** to perform the data scraping from all AROL Group websites (only the pages in English language), with the goal of gathering the filtered and structured data needed for training and validating the LLM (*Large Language Model*) of the chatbot.

Specifically, such work is done by the code written in the file "my_spider.py", which:

1. Collects and organizes structured web content
2. Turns HTML pages into clean and usable representations (text filtered from unnecessary tags by the application of the *BeautifulSoup* transformer)
3. Generate an easily analysable output ("output.jsonl" file) from which to build models based on real data

Note that before deciding to use the Scrapy framework, we made some web scraping tests using **Firecrawl**, an API service that takes a URL, crawls it, and converts it into clean markdown, to allow the gathering of structured data (see file "WebScraper.ipynb").

Parameters Configuration

For implementing this process, we set the **parameters** of the crawler as follows:

```
    custom_settings = {
        "DEPTH_LIMIT": 3,
        "ROBOTSTXT_OBEY": True,
        "USER_AGENT": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36",
        "DOWNLOAD_DELAY": 2,
        "CONCURRENT_REQUESTS_PER_DOMAIN": 8,
        "LOG_LEVEL": "INFO",
        "FEEDS": {
```

```
            "output.jsonl": {
                "format": "jsonlines",
                "encoding": "utf8",
                "overwrite": True
            }
        }
    }
```

- “DEPTH_LIMIT: 3"
  This is used to define the maximum crawling depth with respect to the starting URLs (i.e. https://www.arol.com/arol-canelli), indicating that you want the crawler to analyse the home pages and follow the links for up to 3 levels. In other words, this parameter allows you to control the crawler’s range of action, preventing it from exploring an excessive number of pages, which could make the process inefficient, especially for a relatively simple chatbot.
- “ROBOTSTXT_OBEY: True”,
  “USER_AGENT: "Mozilla/5.0 […] Safari/537.36"”,
  “DOWNLOAD_DELAY: 2”,
  “CONCURRENT_REQUESTS_PER_DOMAIN: 8”
  They ensure that the process is run in accordance with the rules of good manners of scraping, also reducing the risk that the crawler is labelled as aggressive or as a DDoS (*Distributed Denial of Service attack*) and then blocked.

2. Data Preprocessing

Question/Answer Format

As it was said before, the results of the data scraping process performed on AROL Group websites are stored in the file “output.jsonl”, which contains the data we need to train and validate our model.

In this regard, it is observed that scraping produces a partially filtered HTML content of web pages. This format contains a lot of **redundant information** (scripts, styles, metadata) and needs processing to extract:

- Meaningful information
- Content relevant to model training and validation

So, after extracting the relevant content from the websites, we’ve organized the data in a **Question and Answer (QA) format**, which allows:

- **Supervised training:** Teach the model to answer questions based on content provided
- **Targeted inference:** Simulate user-chatbot interactions in specific contexts

Due to the extremely limited amount of data and the complexity of implementing a conversion to the required format, along with the risk of overfitting caused by the similarity of QA pairs, the necessary operations were performed manually using **ChatGPT**.

Specifically, it was enough to provide ChatGPT the content of the file "output.jsonl" requiring it to generate 15 QA pairs at a time. For this reason, it took about 40 iterations to get the pairs reported in "qa.jsonl" and "validation_dataset.jsonl", used respectively to perform chatbot training and validation.

Such datasets are in **JSONL** (*JavaScript Object Notation Lines*) format, convenient for storing structured data that may be processed one record at a time. Indeed, each line represents a JSON object containing a question and its corresponding answer, like shown below:

```
{
  "question": "What does AROL Group do?",
  "answer": "AROL Group specializes in designing, producing, and delivering capping and closure systems, as well as inspection solutions for a wide range of industries."
}
{
  "question": "Where can I download AROL's terms and conditions?",
  "answer": "You can download our terms and conditions from the 'Terms & Conditions' section on the website. Links to the documents are available there."
}
```

Prompt Template

To prepare the data for the fine-tuning process, we used a **prompt template** designed to guide the model by formatting the data for optimal learning. The creation of this template, handled by the formatting_prompt() function, plays a crucial role in the code, since it establishes a clear and structured format with specific instructions for training the model, enabling it to answer targeted questions consistently and effectively.

```
data_prompt = """
You are a customer support assistant for AROL Group, specialized in bottle caps and capping technologies.
Your goal is to provide accurate, clear, and helpful responses about AROL Group's products and processes.

### question:
{}

--- Instructions ---
- Provide a concise and informative response about bottle cap manufacturing or capping technology.
- If technical details or product features are mentioned, explain them simply.
- If concerns are raised, offer relevant recommendations or solutions.
- Keep the answer focused on the specific query.
```

```
### answer:
{}
""".strip()  # Using strip to avoid trailing newlines at the end
```

As shown in the above code, the prompt template defines how questions and the corresponding answers should be linked, based on the principles of clarity and simplicity:

1. A description of the context ("You are a customer support assistant...")
2. A question (placeholder "question")
3. An instruction section (detailed instructions on how to respond)
4. The desired answer (placeholder "answer")

Pandas DataFrame

In the first part of the file "AIChatbot.ipynb", after configuring a working environment for the NLP (*Natural Language Processing*) project based on the available computing device (use GPU with CUDA if available, otherwise the CPU), through the following lines of code the data stored in the files "qa.jsonl" and "validation_dataset.jsonl" are read and converted into **Pandas DataFrames** (stored in the variables "df_training" and "df_validation", respectively), which will be used for training and evaluating the chatbot model.

```
df_training = read_jsonl_to_df("qa.jsonl")

df_validation = read_jsonl_to_df("validation_dataset.jsonl")
```

Pandas is a powerful and popular open-source Python library used for data manipulation and analysis. It is particularly useful for working with structured data, such as spreadsheets or databases, and provides a way to easily manipulate and analyse large amounts of data.

The main data structure in pandas is the **DataFrame**, which is a table-like structure with rows and columns. Once created, the DataFrame can be easily manipulated, cleaned and transformed with a variety of integrated functions and methods.

Hugging Face Datasets

As regards the preparation of data for fine-tuning, before formatting the data according to the prompt template, the Pandas DataFrames corresponding to the training and validation data of the model are converted into **Hugging Face objects Datasets**, with a structure like that of the original DataFrames but optimized for parallel operations and more efficient in data management. The result is a more suitable object for machine learning activities, such as model training and validation.

Indeed, converting data into Hugging Face Datasets offers:

- **Training-specific tools**, such as native support for tokenization with methods like .map()

- **Integrated functionalities for batching** and managing long sequences (useful with models like Llama, GPT)
- **Optimized performance** for scalable NLP pipelines

All the preprocessing steps we've just described are executed on the files "qa.jsonl" and "validation_dataset.jsonl" using the following code:

```
training_data = Dataset.from_pandas(df_training) # Here we convert pandas dataframe into a hugging face dataset object
training_data = training_data.map(formatting_prompt, batched=True) # Here we apply the formatting function to each element of the dataset using map method

validation_data = Dataset.from_pandas(df_validation) # Here we convert pandas dataframe into a hugging face dataset object
validation_data = validation_data.map(formatting_prompt, batched=True) # Here we apply the formatting function to each element of the dataset using map method
```

3. Model fine-tuning

In machine learning, **fine-tuning** refers to changing the parameters of a pretrained model for a specific task or dataset. The model is re-trained with target activity data, maintaining its previous experience.

In this project the main steps for fine-tuning the chatbot model are essentially performed by the code of the file "AIChatbot.ipynb".

Setting up the Pretrained Model

Once the data were ready for the fine-tuning, we set up the pretrained **Llama 3.2** model with 1 billion parameters for fine-tuning, using **LoRA** (Low-Rank Adaptation) to efficiently train only specific parts of the model rather than the entire network, making the process faster and less resource-intensive.

After having realized that by using **1 billion** parameters the result was a chatbot unable to give good answers, we decided to use instead **3 billion** parameters, getting a much better behaviour. Indeed, the number of parameters in a model indicates its size and, to a large extent, its potential for expression or learning: more parameters allow the model to capture more complex structures and relationships in data, making it potentially more **accurate** and **versatile**.

Llama (acronym of **Large Language Model Meta AI**) is a family of large-scale autoregressive language models (LLM) published by Meta AI starting in February 2023. It is designed to understand and generate human language in a natural and coherent way, imitating the way humans speak and write. Specifically, it's based on a specific neural network architecture called **Transformer**, which is one of the most effective for NLP.

The main types of parameters in a model like LlamA include:

- **Weights:** Values that define the importance of connections between neurons or nodes
- **Bias:** Values that allow the model to handle constant output even without input

These parameters are **started randomly** and **updated during training** to allow the model to fit the data.

The pretrained model was loaded with **full precision** (not quantized) for better accuracy, and gradient checkpointing is enabled to manage memory during training. Finally, it prints the number of trainable parameters for verification.

Note that **quantization** is a process that reduces the numerical precision of a model's parameters, turning numbers into a more compact representation. For LLMs, which have millions or billions of parameters, this reduction in precision not only reduces computational weight but also leads to a significant improvement in operational efficiency without unduly compromising performance.

Anyway, the initial attempt at fine-tuning the chatbot failed due to the **4-bit quantization**, which caused significant data loss and rendered the fine-tuning process inefficient. To address this, we decided to **avoid data quantization** altogether, especially since the chatbot is intended to be relatively simple.

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Llama-3.2-3B",
    max_seq_length=max_seq_length,
    load_in_4bit=False,
    dtype=None,
)
```

As shown in the code above, the function FastLanguageModel.from_pretrained():

- Loads a pretrained model called "unsloth/Llama-3.2-3B", which has 3 billion parameters
- The model comes with a **tokenizer**, necessary to convert text into tokens, the numerical representation used by the model

Low-Rank Adaptation

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
     target_modules=["q_proj", "k_proj", "v_proj", "o_proj","gate_proj", "up_proj",
"down_proj",],
```

```
    use_rslora=True,
    use_gradient_checkpointing="unsloth",
    random_state = 32,
    loftq_config = None,
)
```

The call of the function FastLanguageModel.get_peft_model() applies the **PEFT** (*Parameter Efficient Fine-Tuning*) method to the existing model to optimize the fine-tuning process. Instead of training the whole model (very expensive in terms of computation and memory), this technique will only update specific parts.

Precisely, the applied PEFT is **LoRA**, which, at its core, tackles the problem of fine-tuning large models by updating only a tiny fraction of the model's parameters. Instead of retraining the entire model, LoRA introduces a lightweight way of adapting it using low-rank updates.

It leaves most of the model untouched, freezing the original weights (the model's learned knowledge). Instead, it focuses on adjusting these weights using much smaller matrices, which handle the tweaks needed to adapt the model for the new task, all while keeping the original structure intact.

Supervised Training and Validation

To fine-tune the chatbot, the **SFTTrainer**, a trainer specifically optimized for the supervised fine-tuning of large language models (LLMs), was used. The training process utilized the formatted dataset and leveraged all prior configurations to ensure efficient fine-tuning.

Performance Evaluation and Checkpoints

Once the trainer configuration is complete, it is used to perform training (with performance metrics monitoring) and validation with corresponding datasets. As shown in the figure below, we used the validation dataset evaluation to calculate some metrics, such as **loss** and **perplexity**, calculated as $exp(loss)$.

Such metrics are computed and logged every **200 steps**, to ensure a compromise between regular **monitoring** of model performance and **computational efficiency** (no overloading of computational resources).

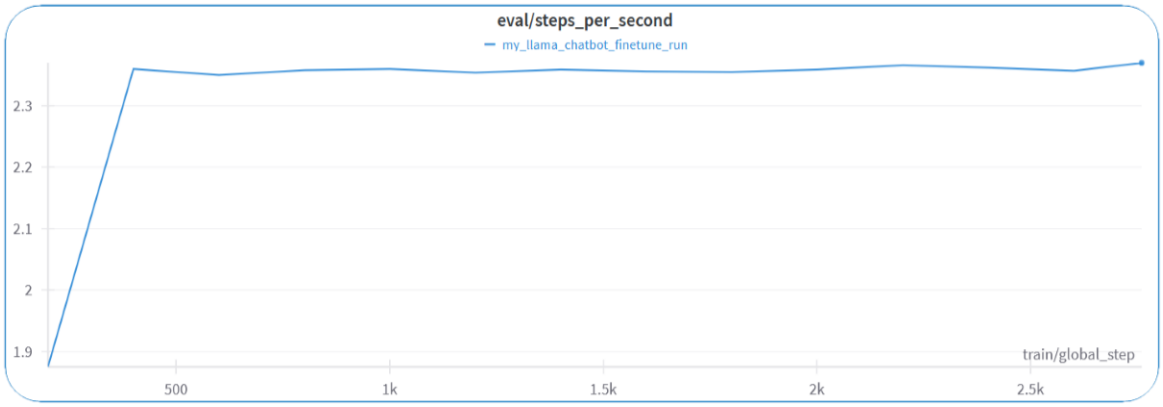
Note that each **step** corresponds to processing a batch of data from the training dataset, loss calculation, backpropagation to calculate gradients, and updating model parameters.

Based on the loss values, we decided to store the model at three specific **training checkpoints**, as they are representative of average cases: checkpoint-400, checkpoint-1200, checkpoint-2000.

[2640/2760 1:06:22 < 03:01, 0.66 it/s, Epoch 37.71/40]

Step	Training Loss	Validation Loss
200	0.196800	0.323974
400	0.121500	0.328974
600	0.091100	0.359895
800	0.074300	0.376075
1000	0.064700	0.410452
1200	0.060100	0.424658
1400	0.057800	0.409700
1600	0.053100	0.431398
1800	0.049700	0.458669
2000	0.048300	0.508953
2200	0.048000	0.532966
2400	0.047000	0.546369
2600	0.046500	0.556386

```
Unsloth: Not an error, but LlamaForCausalLM does not accept `num_items_in_batch`.
Using gradient accumulation will be very slightly less accurate.
Read more on gradient accumulation issues here: https://unsloth.ai/blog/gradient
```



4. Chatbot Inference and Deploying

Hosting Platform

Initially, we used **Google Colab**, which provided free GPUs with a RAM's limit of 15 GB. However, as the fine-tuned model required more GPU resources, we transitioned to **Hugging Face**. While the model performed well, its increased complexity led to significantly longer inference times.

On the **free Hugging Face plan**, which we considered for hosting the chatbot, no GPU was available. Although CPU usage was an option, response times were excessively long (e.g., 700 seconds for a simple query), and the GPU was necessary for accurate answers.

To address these limitations, we finally opted for the **Hugging Face pro version**, enabling the chatbot to respond in about 15 seconds, most of which is spent loading the fine-tuned model.

Hugging Fase Hub Configuration

The fine-tuned model, together with the tokenizer is saved and uploaded to **Hugging Face Hub** to facilitate easy deployment and sharing.

The configuration defined with the code of the file "adapter_config.json" instructs the runtime on how to use the optimized model in the **deployment phase**, taking advantage of the weights and optimizations generated during the fine-tuning. It ensures that the model:

- Keep the customization obtained
- Functions efficiently in real inference scenarios
- Be ready for use via the interfaces of Hugging Face or other related libraries

Inference Mode

Now that the model is trained, it's ready to assist users with their inquiries about AROL Group products and services. In the **inference mode** (the process by which the LLM knowledge is used to answer questions or solve problems, allowing to make more likely decisions), the model leverages the knowledge gained during fine-tuning to generate informative and helpful responses.

Users will input their questions or requests related to AROL Group's offerings and the model processes the user's input, generating a response based on the information it has learned. These responses will be:

- **Domain-Specific:** The model is tailored to AROL Group's expertise, focusing on bottle caps, capping technologies, and related services
- **Informative and Accurate:** Responses are designed to provide clear, relevant, and precise answers, leveraging the domain-specific data used in training

Tokens and Tokenizers

In the context of NLP, a **token** is an elementary unit of text that a model uses to process linguistic information (e.g.: words, parts of words, characters, special sequences).

In the case of pretrained models based on transformers, such as those on Hugging Face, tokens are usually represented as **numbers** (IDs) usable by the model, through a **tokenization** process that converts the original text into this numerical representation.

In the file "tokenizer.json" there is the definition of some special tokens used to improve the chatbot's ability to recognize conversation boundaries and manage specific contexts, allowing the model to learn unique language patterns, suitable for AROL Group.

When the chatbot generates an output sequence (i.e.: a response), special tokens are eventually used as part of the resulting text. The "tokenizer_config.json" file ensures that these tokens are decoded correctly.

The file "special_tokens_map.json" defines a mapping between the types of special tokens standardized (used internally by the model or framework) and their actual representatives in the tokenizer. It is crucial for the proper functioning of a model using special tokens.

Retrieval-Augmented Generation

In the code of the file "app.py", to enhance the chatbot's performance and accuracy, we implemented **RAG (Retrieval-Augmented Generation)**. This technique combines the retrieval of relevant information from a knowledge base with the natural language generation capabilities of an AI model. RAG serves as the foundation for integrating **Pinecone**, which supports efficient and scalable vector-based retrieval.

The key benefits of RAG are the following:

- **Improved Accuracy:** Responses are enriched with up-to-date, relevant information from external knowledge bases, rather than relying solely on the model's pretrained knowledge
- **Customization:** Answers are tailored with organization-specific or domain-specific information
- **Flexibility:** New documents or updates to the knowledge base can be added without the need to re-train the LLM

By incorporating RAG, the LLM dynamically adjusts its **probability distribution** for generating text. Instead of relying solely on pretrained weights or generalized patterns, the retrieved documents focus the probability space on specific, contextually relevant terms and phrases. This ensures that the model generates responses that are not only accurate but also directly aligned with the content retrieved from the knowledge base.

In our model RAG works as follows:

1. A user query is processed and converted into a vector representation using an embedding model
2. The Retriever compares this query vector with vectors stored in the knowledge base (indexed in Pinecone)
3. The most relevant documents (top 5) are retrieved based on similarity
4. The LLM generates a contextualized and precise response by incorporating the retrieved documents

Generation Pipeline

Pinecone is used as the **vector storage and search service** to enable fast and effective retrieval during the RAG process. It stores numerical representations of content that capture its semantic meaning, allowing efficient vector similarity searches.

The chatbot utilizes a combined model with a tokenizer to generate responses that are:

- Clear and concise (token limit: 150)

- Contextually enriched using retrieved external information

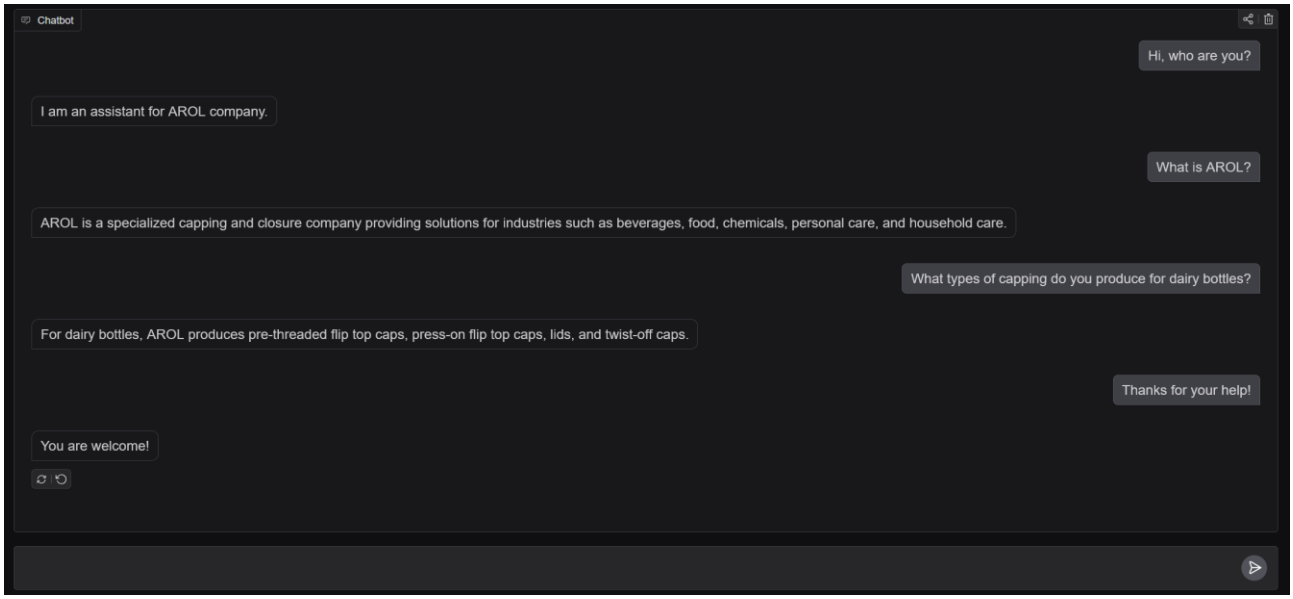
By leveraging RAG and Pinecone, the chatbot achieves higher accuracy, improved domain-specific customization, and flexible updates to its knowledge base.

The **text generation pipeline** uses the combined model with the tokenizer to generate textual responses. Note that the token limit is set to 150, enough to provide clear but concise answers.

Deploying

The chatbot is finally deployed of **Hugging Face Spaces** using **Gradio**, an open-source Python package that allows to quickly build a demo or web application for a machine learning model. We used it to create a chat-based and user-friendly interface accessible via browser, which provides:

- Input/output in real time
- Extended logging to track messages and possible errors



5. Operational Appendix: Chatbot Installation and Configuration

Installation

1. Clone the repository:

```
git clone https://github.com/MelDashti/Arol-ChatBot.git
cd Arol-ChatBot
```

2. Install dependencies for deployment:

```
pip install -r app/requirements.txt
```

Usage

Interact with the Chatbot on Hugging Face Spaces:

Chatbot Demo: https://huggingface.co/spaces/Meldashti/unsloth-llama-3-8b-bnb-4bit?logs=container

Local Deployment (Optional)

1. Set environment variables:

```
export PINECONE_API_KEY="your_pinecone_api_key"
export PINECONE_INDEX="arolchatbot"
```

2. Modify 'app.py' (Comment out the Hugging Face Spaces-specific line):

```
# @spaces.GPU  # Comment this line when running locally
def chat(message, history):
    ...
```

3. Run the application locally:

```
cd app
python app.py
```

Note: Local deployment requires an active Pinecone account and internet access to load models from Hugging Face.

6. Final notes

Authors

Meelad Dashti s328715@studenti.polito.it
Mohammad Hakimi s326334@studenti.polito.it
Federico La Macchia s333959@studenti.polito.it

Related links

Project Repository (Code & Documentation): https://github.com/MelDashti/Arol-ChatBot
Fine-Tuned Model on Hugging Face Hub: https://huggingface.co/Meldashti/chatbot/tree/main
Chatbot demo: https://huggingface.co/spaces/Meldashti/unsloth-llama-3-8b-bnb-4bit?logs=container